DAT42-1

Machine Learning II

Supervised models

Tree-based models, tuning, and end-to-end supervised pipelines.

Albert School — 22 hours

Contents

1 Decision trees

A **decision tree** is a supervised model that predicts by recursively splitting the feature space. Each internal node tests one feature against a threshold, each branch follows the outcome of that test, and each leaf carries a prediction. The same structure serves two tasks:

- A **decision tree classifier** predicts a class label; the leaf prediction is the majority class of the training examples that reach the leaf.
- A **decision tree regressor** predicts a real number; the leaf prediction is the mean target of the training examples that reach the leaf.

1.1 Splitting criteria

A tree is grown by choosing, at each node, the feature and threshold that best split the data according to a **splitting criterion**. For a node containing training examples with class proportions p_k (the fraction of class k at the node), classification uses one of:

$$\text{Gini} = 1 - \sum_{k=1}^{K} p_k^2 \qquad \text{Entropy} = -\sum_{k=1}^{K} p_k \log_2 p_k$$

Both measure **impurity**: they are 0 when the node is pure (all one class) and maximal when classes are evenly mixed. The chosen split is the one that most reduces the weighted impurity of the child nodes.

For regression the criterion is the **mean squared error** (MSE) of a node,

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \bar{y})^2,$$

where $\bar{y}$ is the mean target at the node; the split chosen is the one that most reduces the weighted MSE of the children.

1.2 Visualization

A fitted tree is **visualized** by drawing its nodes top to bottom: each internal node shows its splitting feature and threshold, its impurity, and the sample counts per class; each leaf shows

its prediction. Reading the path from the root to a leaf gives the sequence of conditions that produces a prediction, which makes a single tree interpretable.

2 Random forests

A single deep tree fits the training data closely but is **high variance**: small changes in the data produce a very different tree. A **random forest** reduces this variance by averaging many trees.

2.1 Bagging

Bagging (bootstrap aggregating) trains each tree on a **bootstrap sample** — a sample of the training set drawn with replacement, of the same size as the original. A random forest additionally restricts each split to a random subset of the features, which decorrelates the trees. Predictions are combined by **majority vote** (classification) or **averaging** (regression) across all trees.

2.2 Variance reduction

Averaging B identically distributed predictions, each with variance σ^2 and pairwise correlation ρ, gives variance

$$\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2.$$

Increasing the number of trees B shrinks the second term, and decorrelating the trees (random feature subsets) shrinks ρ. The bias is unchanged, so bagging **reduces variance without increasing bias**.

2.3 Feature importance

A random forest reports **feature importance** by summing, for each feature, the total impurity reduction (Gini, entropy, or MSE) contributed by all splits on that feature, averaged over the trees. A larger value means the feature was more useful for splitting. Importances are interpreted as a ranking of which features drive the model's predictions, not as causal effects.

3 Gradient boosting

Gradient boosting also combines many trees, but builds them **sequentially**: each new tree is fitted to the errors (the residuals, i.e. the negative gradient of the loss) left by the trees built so far, and is added to the ensemble with a small weight. **XGBoost** and **LightGBM** are the standard implementations.

3.1 How boosting differs from bagging

- **Bagging** (random forest) trains trees **in parallel and independently** on bootstrap samples, then averages them to **reduce variance**.
- **Boosting** trains trees **sequentially**, each correcting the previous ones, to **reduce bias**. Boosted trees are typically shallow.

3.2 Key hyperparameters

- **Number of trees** (`n_estimators`): more trees lower training error but risk overfitting.
- **Learning rate** (the weight on each new tree): smaller values need more trees but generalize better.
- **Maximum tree depth**: controls the complexity of each tree.
- **Subsampling** of rows and columns: introduces randomness and regularizes.

The learning rate and number of trees are tuned together: lowering the learning rate while raising the number of trees usually improves generalization.

4 Hyperparameter tuning

Hyperparameters are not learned from the data; they are searched over a grid of candidate values, each evaluated by cross-validation.

- **Grid search** evaluates **every combination** of a discrete set of candidate values. It is exhaustive but its cost grows multiplicatively with the number of hyperparameters. Prefer it when there are few hyperparameters and each has few candidate values.
- **Randomized search** samples a fixed number of random combinations from the hyperparameter space. It covers a wider range at a fixed budget. Prefer it when there are many hyperparameters, or continuous ranges, so an exhaustive grid is infeasible.

5 Feature scaling

Feature scaling puts features on a comparable scale:

- **StandardScaler** subtracts the mean and divides by the standard deviation,

$$x' = \frac{x - \mu}{\sigma},$$

  giving mean 0 and standard deviation 1.
- **MinMaxScaler** rescales to a fixed range, usually $[0, 1]$,

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}.$$

Scaling is **required** by algorithms that depend on the magnitude of features — those using distances, gradient descent, or regularization (e.g. logistic regression, SVMs). It is **not required** by tree-based models (decision trees, random forests, gradient boosting), because their splits depend only on the order of feature values, which scaling does not change. The scaler is fitted on the training set only and then applied to the test set, to avoid data leakage.

6 Advanced feature engineering

Feature engineering builds new input features from raw variables:

- **Polynomial features**: add powers of a feature ($x^2, x^3, ...$) to let a model capture curved relationships.
- **Interaction terms**: add products of features ($x_1 x_2$) to capture effects that depend on two features jointly.
- **Target encoding**: replace a categorical value by the mean target for that category. It is compact for high-cardinality categories but leaks the target, so it must be computed on the training folds only (e.g. with cross-fold encoding) to avoid leakage.
- **Date decomposition**: split a date into components (year, month, day of week, hour, is-weekend, etc.) so that seasonal and cyclic structure becomes available to the model.

7 Class imbalance

When one class is far rarer than the other, a model can score well on accuracy by ignoring the minority class. Three techniques address this:

- **Class weighting**: weight the loss so that errors on the minority class count more. It requires no resampling but assumes the model accepts class weights.
- **SMOTE** (Synthetic Minority Over-sampling Technique): generate synthetic minority examples by interpolating between existing minority points. It balances the classes but can create unrealistic points and must be applied to the training set only.
- **Threshold adjustment**: keep the model unchanged and lower the decision threshold so more instances are predicted as the minority class. It is simple and post-hoc but trades precision for recall.

Each trades off differently: weighting and SMOTE change training, threshold adjustment changes only the decision rule; SMOTE adds synthetic data while the other two do not.

8 End-to-end pipeline

A complete supervised project chains the steps above into a single reproducible pipeline:

1. Split the data into training and test sets.
2. Preprocess: encode and engineer features, scale numeric features for the algorithms that require it, handle class imbalance — all fitted on the training set only.
3. Fit the candidate models.
4. Tune hyperparameters by cross-validation (grid or randomized search).
5. Evaluate on the held-out test set and produce a **written evaluation report**.

9 Model comparison

The course culminates in **comparing three model architectures** on the same problem — typically a single tree, a random forest, and a gradient boosting model — evaluated with the same metric on the same test set. The **final choice** is justified by weighing predictive performance against cost, interpretability, and training and inference time, and the decision is **documented** in the evaluation report.